

TREES :

Tree non-linear data structure

→ data nodes hote hai.

→ Nodes ek hierarchical structure me connected hote hai.

① Parent to multiple childrens

Terms :

① Node → Data box

② Root → Sabse upr wala node

③ Parent → Jo upr connect

④ child → jo neche ho

⑤ leaf → node with no child

⑥ level → root se distance

⑦ height → longest path to leaf

Types

① → Binary tree

② → Binary Search tree (BST)

③ → Complete Binary tree

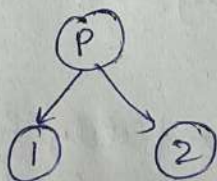
④ Balanced B T

BINARY TREE

① Every node has maximum of 2 childrens
(left & right)

```
struct treeNode {  
    int val;  
    treeNode* left;  
    treeNode* right;  
};
```

```
TreeNode(x) {  
    val = x;  
    left = NULL;  
    right = NULL;  
}
```



BFS → breadth
first search

TRAVERSAL →

ke 2 types hai :

(i) DFS & BFS

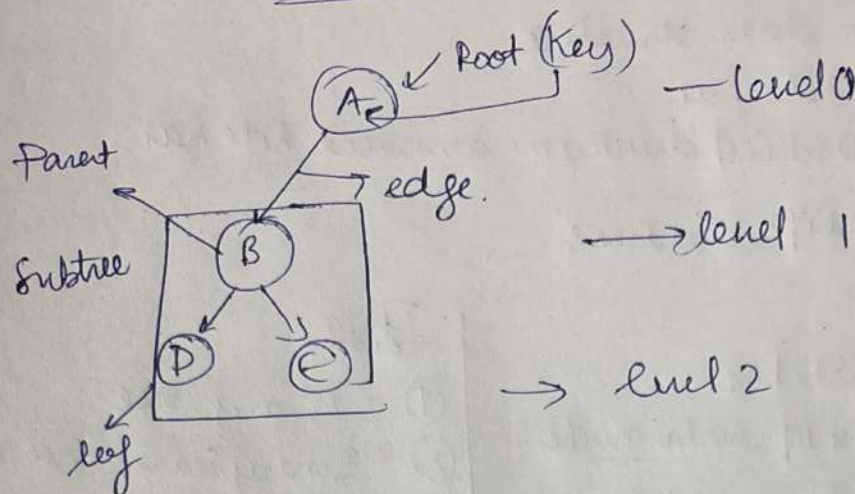
DFS → hum aek node ko
pkd ke neche tk
jate hai fir return krte hai

INORDER

PREORDER

POSTORDER

TREE



- ① general tree $\rightarrow$ ko limits nahi childrens ki.
Rare in interviews
- ② Binary tree $\rightarrow$ Max 2 childrens
 $\rightarrow$ left & right
- ③ BST $\rightarrow$ Binary Search tree
left < Root < right
Sorted Nature
- ④ Complete Binary tree - Sab levels
completely filled
 $\rightarrow$ last level left s fill.
- ⑤ perfect Binary tree $\rightarrow$ Sab levels completely filled
leaf nodes same level pe.
- ⑥ Balanced Binary tree $\rightarrow$ height difference ≤ 1
- ⑦ Degenerated / Skewed tree $\rightarrow$ linked list jesa hota hai
 $\rightarrow$ Worst case
- ⑧ Heap $\rightarrow$ Complete tree / (Min heap) / max heap

DFS → Depth first Search :

Inorder → L → Root → right

Preorder → Root → L → Right

Postorder → L → Right → Root

Struct Node {

int data;

Struct Node* left, right;

Node(int data)

{

this->data = data;

left = right = NULL;

}

* void inorder (Node* node)

{

if (node == NULL)

return;

inorder (node->left);

printf ("node->data");

inorder (node->right);

}

* void preorder (Node* node)

{ if (node == NULL)

return;

printf ("Root->data");

preorder (node->left);

preorder (node->right);

}

* void postorder (Node* node)

{ if (node == NULL)

return;

postorder (node->left);

postorder (node->right);

printf ("%d", node->data);

① Inorder : BST me inorder is Sorted

→ middle mai root comes

→ left subtree comes before.

BST to sorted array → Inorder

Validate BST → Inorder → check if sorted

Kth small/largest → Inorder

range query is → Sorted order

PREORDER :

- Root → left → right
- Tree ka structure capture karta hai.
- Copy/serialise me use

- ① tree copy → root
- ② Tree serialise/de → structure preserved
- ③ parent before child → preorder

Root k kam → preorder
Children → Postorder
Sort/kth/ → Inorder

POSTORDER

- Root sbse last mai.
 - left → right → root
 - Children phle process hote hai.
- * ① Tree delete krna
 - ② Bottom up calculation
 - ③ Expression tree

DFS

```
void DFS (Node* root)
{
    if (root == Null) return;
    // pre
    DFS (root->left);
    // in
    DFS (root->right);
    // post
}
```

Struct Node &

```
int val;
Node* left;
Node* right;
public
Node (int x) {
    this->val = x;
    left = right = Null;
}
```

MAX Depth/Height

```
0 → level 0 ①+
if (root == Null) return 0;
return 1 + max (mdepth (left), mdepth (right));
```

path from Root to leaf :

- ① if (root == Null) return;
- push . push_back (root->val);
- if (root->right == Null || left == Null)
- dfs (root->left)
- dfs (root->right)
- path . pop_back ();

bool isSame (*a, *b) {

if (a == NULL && b == NULL) return true;

if (a == NULL || b == NULL) return false;

if (a->val != b->val) return false;

return isSame(a->left, b->left) &&
isSame(a->right, b->right);

BFS :

V < V < int > levelOrder(TreeNode* root)

if (root == NULL) return;

Queue < TreeNode > q;

q.push(root);

while (!q.empty())

~~TreeNode front = q.front();~~ int s = q.size();

~~q.pop();~~

~~level.push(q->val);~~

as.push(level);

V < V < int > level,

for (int i = 0; i < s; i++)

TreeNode* temp = q.front(); q.pop();

level.pushback(temp->val);

if (temp->left) q.push(temp->left);

if (temp->right) q.push(temp->right);

TreeNode* insert BST (root, key)

if (root == NULL) return new TreeNode(key);

if (key < root->val)

root->left = call karo

else root->right = call karo

Search

if (root == NULL ||
root->val == key)

return true;

if (key > root->val)

// right mai
hogi.

else

// left mai hogi

// (search(root->left));

bool validate (root, long long minval, long long maxval)

if (root == null) return true;

if (root->val > minval && root->val <

Real BST mai :

* right subtree ka har node > root hona chahia

* left subtree ka har node < root hona chahia

① Approach Inorder mai hamesha sorted rehta hai

to

if (root == null) return true;

if (!isvalid (root->left)) return false;

if (root->val <= prev) return false;

prev = root->val;

return (isvalid (root->right));

kyunki hum left s
are hai to koi option
hi ni ki prev humse
bada ho